

Industrial Internship Report on

"Python Programming"

Covering Four Projects:
URL Shortener | File Organizer | Password Manager | Quiz Game

Prepared by

M.SHEHANA SAFREEN

244002 | K.L.N College of Engineering

Internship facilitated by
upskill Campus (USC) & The IoT Academy
in association with UniConverge Technologies Pvt Ltd (UCT)

Duration: 6 Weeks | Domain: Python Development

Executive Summary

This report documents the Industrial Internship facilitated by upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt Ltd (UCT). The internship spanned six weeks and was centered on building four fully functional Python application projects, each targeting a distinct problem domain.

The four projects developed during this internship are: (1) a Flask-based URL Shortener web application with analytics, (2) a command-line File Organizer that automates directory management, (3) a cryptographically secure Password Manager using Fernet encryption and PBKDF2HMAC key derivation, and (4) an interactive terminal-based Quiz Game with a scoring and leaderboard system.

Each project progressed through two weekly sprints — covering setup and core functionality in Week 1, followed by refinement, feature addition, testing, and documentation in Week 2. Together, these projects provided hands-on exposure to web development, file system operations, cryptography, database handling, and game logic in Python.

This internship was an invaluable experience that bridged academic Python knowledge with practical software engineering, preparing me for real-world development challenges in the industry.

TABLE OF CONTENTS

1	Preface	4
2	Introduction	5
2.1	About UniConverge Technologies Pvt Ltd	5
2.2	About upskill Campus (USC)	5
2.3	Objectives of the Internship	5
2.4	References	6
2.5	Glossary	6
3	Problem Statement	7
4	Existing and Proposed Solution	8
5	Proposed Design / Model	9
5.1	URL Shortener — System Flow	9
5.2	File Organizer — Process Flow	9
5.3	Password Manager — Security Architecture	9
5.4	Quiz Game — Game Loop Design	9
6	Performance Test	10
7	My Learnings	11
8	Future Work Scope	12

1 Preface

This report is a comprehensive account of six weeks of intensive Python application development carried out as part of the Industrial Internship program organized by upskill Campus and The IoT Academy, in association with UniConverge Technologies Pvt Ltd. The internship was designed to bridge the gap between theoretical programming knowledge and real-world software development practice.

Practical internship experience has become an indispensable component of a technology career today. Employers increasingly expect fresh graduates to arrive with not only conceptual understanding but also hands-on project experience. This internship provided exactly that — a structured, industry-guided environment to design, build, and iterate on real software products within tight weekly deadlines.

Over the course of six weeks, I developed four Python projects of increasing complexity: a URL Shortener web application, a File Organizer CLI tool, a Password Manager with encryption, and a Quiz Game with leaderboard functionality. Each project was scoped, developed, and reviewed in two-week sprints, mirroring real software development cycles.

The program was structured to provide weekly milestones with mentor feedback, encouraging iterative improvement — a habit central to professional software engineering. Working within these constraints taught me to prioritize features, debug under pressure, and write cleaner, more maintainable code with each passing week.

I would like to sincerely thank the mentors and coordinators at UCT and upskill Campus for their continuous guidance and constructive feedback throughout the program. Their real-world perspective helped me see beyond textbook solutions and think about the practical implications of my design decisions.

To my peers and juniors who will undertake this internship in the future: approach each project with genuine curiosity. The hours spent debugging and refactoring are never wasted — they are where the most durable learning happens. Commit to the process, and the results will follow.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a technology company established in 2013, operating in the Digital Transformation domain. UCT develops industrial solutions with a prime focus on sustainability and Return on Investment (RoI). The company leverages cutting-edge technologies including the Internet of Things (IoT), Cybersecurity, Cloud Computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, and modern frontend frameworks.

UCT's core product offerings include UCT Insight — an IoT platform enabling device connectivity via MQTT, CoAP, HTTP, and Modbus TCP protocols — and FactoryWatch, a Smart Factory platform providing OEE monitoring, predictive maintenance, and digital twin capabilities. UCT is also an early adopter of LoRaWAN technology, delivering solutions in Agritech, Smart Cities, and Industrial Monitoring.

2.2 About upskill Campus (USC)

upskill Campus (USC) is a career development platform that delivers personalized executive coaching in a more affordable, scalable, and measurable way. In collaboration with The IoT Academy and UniConverge Technologies, USC facilitates the smooth execution of the complete internship process — from onboarding and project allocation to weekly mentorship and final evaluation.

The IoT Academy is the EdTech division of UCT, running executive certification programs in collaboration with EICT Academy, IIT Kanpur, IIT Roorkee, and IIT Guwahati across multiple technology domains.

2.3 Objectives of this Internship Program

The primary objectives of this internship program were:

- To gain practical, hands-on experience working on real software projects in an industry-guided environment.
- To solve real-world problems by designing, implementing, and iterating on Python applications across multiple domains.
- To improve job readiness through exposure to professional software development practices including modular code design, version control, testing, and documentation.
- To deepen domain understanding in areas such as web development (Flask), file system management, cryptography, and game logic.
- To develop personal and professional skills including time management, communication, problem-solving under constraints, and iterative improvement based on mentor feedback.

2.4 References

- [1] Flask Official Documentation — <https://flask.palletsprojects.com>
- [2] Python cryptography library — <https://cryptography.io>
- [3] Real Python Tutorials — <https://realpython.com>
- [4] Python Official Documentation — <https://docs.python.org/3>

[5] Bootstrap 5 Documentation — <https://getbootstrap.com/docs/5.0>

[6] OWASP Password Storage Cheat Sheet — <https://cheatsheetseries.owasp.org>

2.5 Glossary

Term	Acronym / Meaning
Flask	A lightweight Python web framework used for building web applications
SQLite	A lightweight, serverless relational database engine built into Python
Fernet	Symmetric authenticated encryption built on AES-128-CBC from the cryptography library
PBKDF2HMAC	Password-Based Key Derivation Function 2 — derives an encryption key from a master password
CLI	Command-Line Interface — a text-based interface for interacting with software
shutil	Python standard library module for high-level file operations (copy, move, delete)
os.walk()	Python function for recursively traversing directory trees
argparse	Python module for building command-line argument parsers
secrets	Python module for generating cryptographically secure random values
colorama	Python library for adding ANSI color codes to terminal output
API	Application Programming Interface — a set of defined interactions between software components
UCT	UniConverge Technologies Pvt Ltd
USC	upskill Campus
IoT	Internet of Things — network of physical devices connected to collect and exchange data

3 Problem Statement

The internship was structured around four distinct problem statements, each representing a common real-world software challenge. Together they covered the breadth of Python application development — web services, file automation, security, and interactive applications.

Project 1: URL Shortener

Long URLs shared across platforms are difficult to read, remember, and embed in text-constrained environments. There is a need for a web-based service that accepts a long URL, generates a unique short code, and redirects users to the original URL when the short link is accessed. The system must also prevent duplicate entries, handle invalid URLs gracefully, and provide basic analytics on link usage.

Project 2: File Organizer

Users frequently accumulate large numbers of unsorted files in download folders and working directories, making it difficult to locate specific files efficiently. There is a need for an automated CLI tool that scans a given directory, categorizes files by type, and moves them into appropriately named subfolders — with support for recursive scanning, dry-run previewing, and detailed logging.

Project 3: Password Manager

Managing passwords for multiple online accounts is a significant challenge. Reusing passwords is a security risk, while maintaining unique strong passwords without a secure storage system is impractical. There is a need for a local password manager that securely encrypts and stores credentials, derives encryption keys from a master password without storing the key itself, and allows users to add, retrieve, and generate strong passwords through a CLI interface.

Project 4: Quiz Game

Interactive learning tools improve knowledge retention through active recall. There is a need for a terminal-based quiz game that presents questions from a database across multiple categories, enforces per-question time limits, provides real-time color-coded feedback, and maintains a persistent leaderboard to motivate continued engagement.

4 Existing and Proposed Solution

4.1 Existing Solutions & Their Limitations

Several existing tools address the individual problems targeted by these projects:

- URL Shorteners: Commercial services like Bitly and TinyURL exist but require account creation, have rate limits, and offer no self-hosted analytics control.
- File Organizers: GUI-based tools like Hazel (macOS) exist but are platform-specific, paid, and not programmable by the average developer.
- Password Managers: Tools like LastPass and 1Password are feature-rich but cloud-dependent, subscription-based, and opaque in their security implementations.
- Quiz Tools: Google Forms and Kahoot serve quiz needs but require internet connectivity and offer no offline, developer-controlled experience.

4.2 Proposed Solutions & Value Addition

The proposed solutions are lightweight, open-source Python applications that give users full control over their data and behavior:

- URL Shortener: A self-hosted Flask web app with SQLite storage, collision-safe code generation, input validation, click analytics, and a Bootstrap UI — deployable anywhere without third-party dependency.
- File Organizer: A cross-platform Python CLI tool with recursive scanning, extensible file-type mappings, dry-run preview mode, and audit logging — far more transparent and customizable than GUI alternatives.
- Password Manager: A local CLI password manager using industry-standard Fernet encryption with PBKDF2HMAC key derivation — no cloud dependency, no subscription, full cryptographic transparency.
- Quiz Game: An offline terminal quiz game with SQLite question storage, category filtering, countdown timers, colorama-powered feedback, and a persistent leaderboard — fully customizable and extensible.

4.3 Code & Report Repository

CodeSubmission(GitHub):

<https://github.com/shehacreate/upskillcampus/tree/main/File%20organiser>

<https://github.com/sheha-create/upskillcampus/tree/main/URL%20Shortener>

<https://github.com/sheha-create/upskillcampus/tree/main/Quiz%20Game>

<https://github.com/sheha-create/upskillcampus/tree/main/Password%20Manager>

Report Submission (GitHub):

<https://github.com/shehacreate/upskillcampus/tree/main/File%20organiser>

<https://github.com/sheha-create/upskillcampus/tree/main/URL%20Shortener>

<https://github.com/sheha-create/upskillcampus/tree/main/Quiz%20Game>

<https://github.com/sheha-create/upskillcampus/tree/main/Password%20Manager>

5 Proposed Design / Model

Each project follows a modular architecture with clear separation of concerns between the interface layer, business logic, and data storage.

5.1 URL Shortener — System Flow

User submits a long URL via the Bootstrap form → Flask validates the URL using `urllib.parse` → `utils.py` generates a 6-character alphanumeric short code using `hashlib` → `db.py` checks for collision and inserts the record into SQLite → the short URL is returned and displayed. On access, `GET /<short_code>` → `db.py` retrieves the original URL and increments the click counter → Flask issues an HTTP 302 redirect. The `/stats/<short_code>` route reads the click count and creation date from the database and renders an analytics card.

Modules: `app.py` (Flask routes), `db.py` (SQLite CRUD), `utils.py` (code generation, URL validation).

5.2 File Organizer — Process Flow

User invokes the CLI with a target directory path and optional flags (`--recursive`, `--dry-run`) → `argparse.py` parses arguments → `organizer.py` calls `os.walk()` to traverse the directory → each file's extension is passed to `categories.py` to determine its target folder → `create_folders()` creates missing category directories → `move_files()` calls `shutil.move()` within a try-except block → all actions are written to `organizer.log` via the logging module. In dry-run mode, moves are printed but not executed.

Modules: `app.py` (CLI entry point), `organizer.py` (scan + move logic), `categories.py` (extension mapping).

5.3 Password Manager — Security Architecture

On first run, a random salt is generated and stored. The user sets a master password → PBKDF2HMAC derives a Fernet key from the password + salt (key never stored on disk) → a sentinel value is encrypted and stored to verify future logins. On subsequent logins, the derived key decrypts the sentinel — a failure indicates a wrong master password. Passwords are encrypted before being stored in SQLite. The generator uses Python's `secrets` module for cryptographically secure password generation.

Modules: `app.py` (CLI menu), `crypto.py` (Fernet encryption, PBKDF2HMAC), `db.py` (SQLite CRUD), `generator.py` (password generation).

5.4 Quiz Game — Game Loop Design

User selects a category from the menu → SQLite fetches filtered questions → questions are shuffled using `random.shuffle()` → for each question, a countdown timer thread starts (threading module) → the question and options are displayed with `colorama` formatting → user input is validated in a try-except loop → on correct answer, the score increments and green feedback is shown; on wrong answer or timeout, red feedback and the correct answer are shown → after all questions, the final score is calculated and saved to the leaderboard table in SQLite → the leaderboard is displayed.

Modules: `app.py` (entry + menu), `quiz.py` (game loop, timer), `data.py` (question loading), `score.py` (scoring + leaderboard), `db.py` (SQLite).

6 Performance Test

Each project was tested against functional correctness, edge case handling, and usability constraints. The following summarizes the test plan, procedures, and outcomes across all four projects.

6.1 Test Plan / Test Cases

Test Case	Input / Action	Expected Output	Result
TC-01	Submit a valid long URL	Unique short code returned and stored in DB	Pass
TC-02	Submit a malformed URL (no schema)	Error message displayed, no DB write	Pass
TC-03	Access a valid short code	302 redirect to original URL, click count incremented	Pass
TC-04	Submit duplicate long URL	Existing short code returned without duplicate DB entry	Pass
TC-05	Run File Organizer on mixed directory	Files sorted into Images, Documents, Videos, Misc folders	Pass
TC-06	File Organizer with --dry-run flag	Actions printed to console, no files moved	Pass
TC-07	File Organizer on file with PermissionError	Error logged to organizer.log, program continues	Pass
TC-08	Password Manager: correct master password	Credentials decrypted and displayed correctly	Pass
TC-09	Password Manager: wrong master password	Sentinel decryption fails, access denied message shown	Pass
TC-10	Password Manager: generate password	Strong random password with mixed characters generated	Pass
TC-11	Quiz Game: correct answer within time	Score increments, green feedback displayed	Pass
TC-12	Quiz Game: timer expires before answer	Question skipped, red feedback, correct answer shown	Pass

TC-13	Quiz Game: non-numeric input entered	Re-prompt shown, program does not crash	Pass
TC-14	Quiz Game: complete session leaderboard	Score saved to SQLite, leaderboard table displayed	Pass

6.2 Test Procedure

All tests were conducted manually in a local Python 3.11 environment on Windows 11. Flask routes were tested using the browser and curl commands. File Organizer tests were run on a dedicated test directory populated with sample files of various types. Password Manager tests were run in isolation with a fresh database for each test session. Quiz Game tests were performed interactively, with timer behavior verified by allowing questions to naturally expire.

6.3 Performance Outcome

- URL Shortener: All 4 URL-related test cases passed. Average response time for the /shorten endpoint was under 50ms on localhost. The collision detection loop added negligible overhead.
- File Organizer: Correctly sorted 47 test files across 6 categories. The --dry-run mode prevented any unintended file movement. PermissionError handling ensured the tool did not crash on locked files.
- Password Manager: Master password derivation via PBKDF2HMAC added approximately 300ms on first login — expected and acceptable for a security-critical operation. Wrong password detection worked reliably in all test cases.
- Quiz Game: Timer-based question skipping worked correctly with no race conditions observed across 20 test rounds. Leaderboard persistence was verified across multiple game sessions.

7 My Learnings

This six-week internship was one of the most concentrated and meaningful learning experiences of my academic journey. Each project taught me something distinct, and together they gave me a rounded perspective on Python software development.

From the URL Shortener, I learned how web applications are structured using Flask — routing, request handling, database integration, and frontend templating. Dealing with the short code collision issue early on taught me to think about concurrency and data integrity from the very beginning of a project, not as an afterthought.

The File Organizer deepened my understanding of Python's standard library, particularly `os`, `shutil`, and `pathlib`. More importantly, it taught me that robust software must anticipate failure — the `PermissionError` handling and dry-run mode were features I only thought to add after encountering real problems during testing. That lesson — build defensively — will stay with me.

The Password Manager was the most intellectually demanding project. It introduced me to applied cryptography: the difference between hashing and encryption, why keys should never be stored in plaintext, and how PBKDF2HMAC turns a human-memorable password into a cryptographically strong key. The OWASP reading was eye-opening — it made me realize how many ways a seemingly functional security feature can have subtle but critical vulnerabilities.

The Quiz Game brought together everything learned across the previous projects — database design, input validation, modular architecture — while introducing new concepts like threading for the countdown timer and `colorama` for terminal UX. Seeing a genuinely playable application emerge from a few hundred lines of Python was deeply satisfying.

Beyond the technical skills, this internship taught me how professional software is built iteratively — through planning, building, testing, failing, fixing, and refining. Working within weekly deadlines and incorporating mentor feedback mirrored a real agile development cycle, and that experience is something no classroom exercise can replicate.

8 Future Work Scope

Each project has clear pathways for further development. The following outlines the most impactful extensions that could be pursued in future iterations:

8.1 URL Shortener

- User authentication system allowing each user to manage their own shortened links with a personal dashboard.
- Custom alias support so users can define their own short code instead of using an auto-generated one.
- QR code generation for each shortened URL using the qrcode Python library.
- Expiry date functionality so links automatically deactivate after a configurable time period.
- Deployment to a cloud platform (Heroku, Render, or AWS) with a real domain name for public use.

8.2 File Organizer

- A graphical user interface (GUI) built with Tkinter or PyQt to make the tool accessible to non-technical users.
- Undo functionality that records all file movements in a JSON log and allows reverting them with a single command.
- Duplicate file detection using file hashing (MD5/SHA-256) to identify and optionally remove identical files regardless of name.
- Scheduled organization runs using Python's schedule library or OS task schedulers to automate folder cleanup at regular intervals.

8.3 Password Manager

- A secure clipboard integration to copy passwords directly without displaying them on screen.
- Password strength scoring at entry time to warn users when a manually entered password is weak.
- Encrypted export and import functionality for backing up credentials to a portable encrypted file.
- Two-factor authentication (TOTP) support using the pyotp library for an additional layer of security on the master login.

8.4 Quiz Game

- A web-based version using Flask with a multiplayer lobby where friends can compete in real time.
- An admin panel to add, edit, and delete questions from the SQLite database through a graphical interface.
- Difficulty levels (Easy, Medium, Hard) with questions tagged accordingly and separate leaderboards per difficulty.
- Text-to-speech integration using pyttsx3 to read questions aloud for an accessibility-friendly mode.

- API integration with Open Trivia Database (opentdb.com) to dynamically fetch fresh questions instead of relying solely on a static local database.

End of Report